

1. **A sh**t ton of regexes** For each of the following languages over the alphabet $\{0, 1\}$, give a regular expression that describes that language, and briefly argue why your expression is correct.

- (a) All strings that end in **1011**.

Solution: $(0 + 1)^*1011$.

The expression $(0 + 1)^*$ matches a string of **1**'s and **0**'s of arbitrary length. The substring **1011** in the end of the regular expression ensures all matching strings must end with it. ■

- (b) All strings except **11**.

Solution: $\epsilon + 1 + 0 + 00 + 01 + 10 + (0 + 1)^3(0 + 1)^*$

The first part $\epsilon + 1 + 0 + 00 + 01 + 10$ covers all strings up to length 2, explicitly not listing **11**. The expression $(0 + 1)^3(0 + 1)^*$ represents all strings of length three or greater. ■

- (c) All strings that contain **101** or **010** as a substring.

Solution: $(0 + 1)^*(101 + 010)(0 + 1)^*$.

The expression $(0 + 1)^*$ refers to all binary strings of length zero or more. A string with **101** or **010** substring, would have **101** or **010** with any number of pre/post-ceding **0**'s and **1**'s. ■

- (d) All strings that contain **111** and **000** as a subsequence (the resulting expression is long – describe how you got your expression, instead of writing it out explicitly).

Solution: Let S be the set of all strings of length 6 that have the desired property. That is, we have

$$S = \left\{ \begin{array}{l} 000111, 001011, 001101, 001110, 010011, \\ 010101, 010110, 011001, 011010, 011100, \\ 111000, 110100, 110010, 110001, 101100, \\ 101010, 101001, 100110, 100101, 100011 \end{array} \right\}.$$

There are $\binom{6}{3} = 6 \cdot 5 \cdot 4 / 6 = 20$, such strings, and they all belong to our language. For a string s , let $f(s)$ be the regular expression of inserting $(0 + 1)^*$ between any two characters of s , and also in the beginning of s and the end of s . For example, we have $f(01) = (0 + 1)^*0(0 + 1)^*1(0 + 1)^*$. The desired expression is

$$\sum_{s \in S} f(s).$$

■

- (e) The language containing all strings that do not contain 111 as a substring.

Solution: $((\epsilon + 1 + 11)0^+)^*(\epsilon + 1 + 11)0^* = ((\epsilon + 1 + 11)0)^*(\epsilon + 1 + 11)0^*$.

A *run* is a string s is a maximal substring of s , all made of the same character. Given a string w in the language, break it into blocks, where a block starts just before a run of 1s start. For example,

$$010001101011 = 0|1000|110|10|11.$$

Such a block starts with a run of 1s of length zero, one, or two. It is then followed by a run of at least one 0. The only exception is the last block, which does have to end with zeros. ■

- (f) All strings that do *not* contain 000 as a subsequence.

Solution: This is just a complicated way to say that the string can contain at most two 0s. Before, in between and after the zeros, we can have a run of ones. As such, the solution is: $1^*(\epsilon + 0)1^*(\epsilon + 0)1^*$. ■

- (g) Strings in which every occurrence of the substring 00 appears before every occurrence of the substring 11.

Solution: $(10 + 0)^*(1 + 10)^*$.

Indeed, break the string in the language into two parts. The prefix p that contains no consecutive 1s, and the suffix s that contain no consecutive 0s.

The expression $(10 + 0)^*$ includes all strings with no consecutive 1s, and matches p . Similarly, the expression $(1 + 10)^*$ includes all strings with no consecutive 0s. ■

- (h) Strings that do not contain the subsequence 010.

Solution: $1^*0^*1^*$.

The condition implies that there could not be two runs of zeros in the string. ■

- (i) Strings that do not contain the subsequence 0101010.

Solution: $1^*0^*1^*0^*1^*0^*1^*$.

The condition implies that there could be at most three runs of zeros in the string. ■

- (j) Strings that do not contain the subsequence 10.

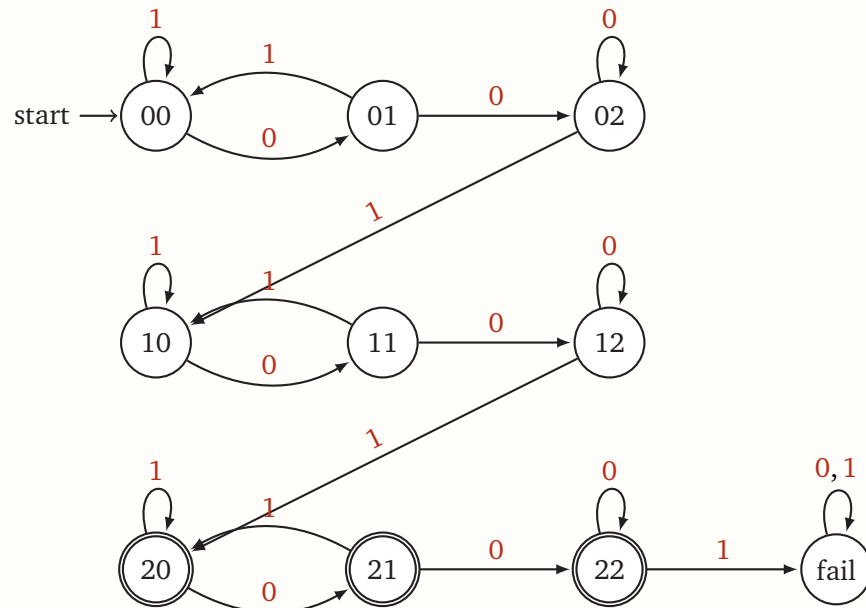
Solution: 0^*1^* .

Such strings can not contain a run of 1s followed by a run of 0s. As such, it can contain only a run of 0s followed by a run of 1s. ■

2. Make some DFAs

- (a) Let L be the set of all strings in $\{0, 1\}^*$ that contain exactly two occurrences of the substring 001 . Describe a DFA that over the alphabet $\Sigma = \{0, 1\}$ that accepts the language L . Argue that your machine accepts every string in L and nothing else, by explaining what each state in your DFA means. (You may either draw the DFA or describe it formally, but the states Q , the start state s , the accepting states A , and the transition function δ must be clearly specified.)

Solution: The following 10-state DFA accepts the language. Every state except fail is labeled with a pair of integers (i, j) , where i is the number of times we have seen the substring 001 and j is the number of 0s (up to 2) we have just read. The machine enters the fail state when it sees 001 for the third time.



- (b) In certain programming languages, comments appear between delimiters such as `/#` and `#/`. Let C be the language of all valid delimited comment strings. A member of C must begin with `/#` and end with `#/` but have no intervening `#/`. For simplicity, assume that the alphabet for C is $\Sigma = \{a, b, /, \#\}$. Give a DFA that recognizes C .

Solution: The six state DFA accepts the language. Since any string must start with `/#`, the A, B states enforce that. After the `/#`, any `#` would change the state to C, which means if the next character is `/`, the string must end with no further characters. The transition from D to fail enforces this.

